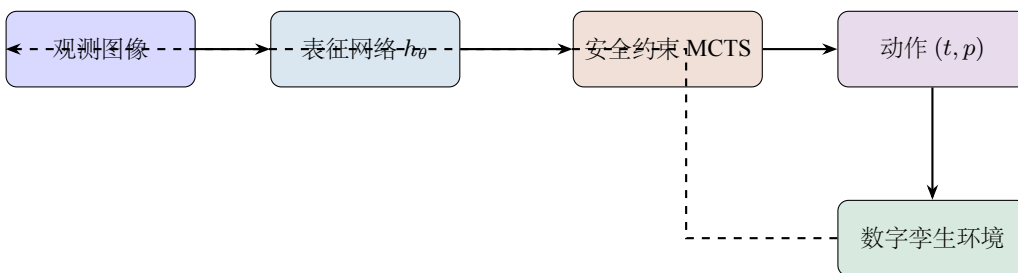


μ -MuZero 项目完整学习手册

面向自主微机器人手术的模型强化学习

安全约束随机变体的深度解析



作者：Yunxiao Ren

帝国理工学院哈姆林机器人手术中心

2026 年 5 月 1 日

目录

1 项目概述与背景	2
1.1 什么是 μ -MuZero?	2
1.2 微尺度带来的三大挑战	2
1.3 μ -MuZero 的核心解决方案	2
1.4 性能基准	2
2 系统整体架构	3
2.1 数据流总览	3
2.2 模块职责	3
3 配置模块详解: config.py	4
3.1 配置类定义	4
3.2 参数分类详解	4
3.2.1 环境参数	4
3.2.2 微物理参数	5
3.2.3 课程学习阶段	5
3.2.4 动作原语	5
4 神经网络模块详解: networks.py	6
4.1 整体架构图	6
4.2 表征网络 h_θ : 观测 $\rightarrow$ 潜态	6
4.2.1 网络结构	6
4.2.2 残差块设计	7
4.2.3 表征网络逐层展开结构	7
4.2.4 残差块内部结构	7
4.2.5 前向传播公式	7
4.3 动力学网络 g_θ : 核心创新	7
4.3.1 为什么需要随机动力学?	7

2 系统整体架构

2.1 数据流总览

μ -MuZero 的完整系统由五个核心模块组成，数据流如下：

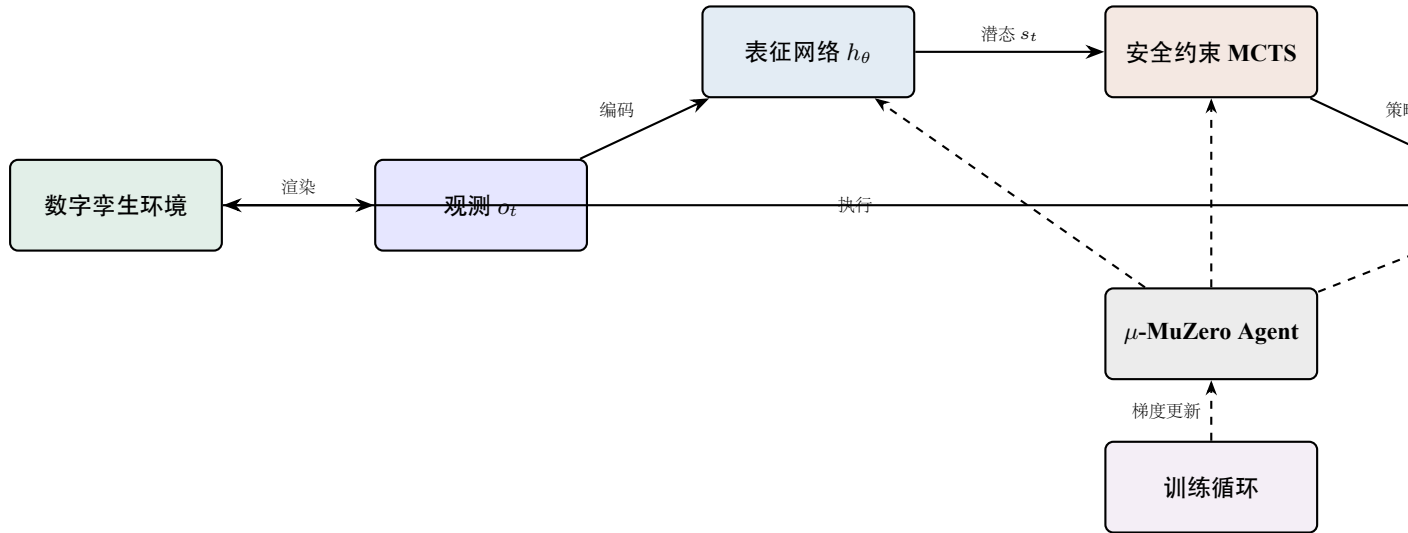


图 1: μ -MuZero 系统数据流图

2.2 模块职责

表 3: 各模块职责说明

模块	文件名	核心职责
配置模块	config.py	集中管理所有超参数与安全约束
神经网络	networks.py	表征/动力学/预测三大网络的前向传播
MCTS 规划器	mcts.py	安全约束下的蒙特卡洛树搜索
数字孪生	environment.py	模拟微尺度物理环境
Agent 接口	mu_muzero.py	整合模型、规划器、环境，实现自博弈
训练脚本	train.py	主训练循环、评估、日志记录
PPT 生成器	ppt_generator*.py	自动生成学术演示文稿

3 配置模块详解: config.py

配置模块使用 Python dataclass 实现, 集中管理所有超参数, 避免分散的魔法数字。

3.1 配置类定义

```
1 from dataclasses import dataclass
2 from typing import Tuple
3
4 @dataclass
5 class MuMuZeroConfig:
6     # 环境参数
7     workspace_size: Tuple[float, float, float] = (100.0, 100.0, 50.0) # um
8     num_traps: int = 3
9     max_laser_power: float = 100.0 # mW (硬安全约束)
10    phototoxicity_budget: float = 5000.0 # mW*ms
11
12    # 微物理参数
13    viscosity: float = 1.0e-3 # Pa*s (水, 20C)
14    temperature: float = 293.15 # K
15    brownian_noise_scale: float = 0.1 # um/sqrt(ms)
16
17    # 神经网络参数
18    latent_dim: int = 256
19    num_res_blocks: int = 8
20    hidden_dim: int = 256
21
22    # MCTS 参数
23    num_simulations: int = 50
24    c_puct: float = 1.25
25    dirichlet_alpha: float = 0.3
26    dirichlet_epsilon: float = 0.25
27
28    # 训练参数
29    batch_size: int = 512
30    lr: float = 1e-3
31    weight_decay: float = 1e-4
32    num_unroll_steps: int = 5
33    td_steps: int = 10
```

3.2 参数分类详解

3.2.1 环境参数

- `workspace_size = (100, 100, 50)`: 三维工作空间 (单位: μm), 对应显微镜视野范围。
- `num_traps = 3`: 同时操控的光阱数量, 决定动作空间维度。
- `max_laser_power = 100.0 mW`: 硬安全约束, 任何时刻激光功率不得超过此值。

- phototoxicity_budget = 5000.0 mW·ms: 软安全约束, 单回合累积光毒性预算。

3.2.2 微物理参数

这些参数直接来自流体力学与统计物理:

- viscosity = 10^{-3} Pa·s: 水在 20°C 的动力粘度。
- temperature = 293.15 K: 绝对温度, 用于计算布朗运动强度。
- brownian_noise_scale = $0.1 \mu\text{m}/\sqrt{\text{ms}}$: 布朗位移的扩散系数标度。

布朗运动的均方位移满足爱因斯坦关系:

$$\langle \Delta x^2 \rangle = 2D\Delta t \quad (1)$$

其中扩散系数 $D = \frac{k_B T}{6\pi\eta r}$, k_B 为玻尔兹曼常数, η 为粘度, r 为粒子半径。

3.2.3 课程学习阶段

```
1 curriculum_stages: Tuple[dict, ...] = (
2     {"name": "single_robot", "success_threshold": 0.7, "max_steps": 50000},
3     {"name": "obstacle_avoidance", "success_threshold": 0.7, "max_steps":
4         200000},
5     {"name": "two_robot_coop", "success_threshold": 0.7, "max_steps": 800000},
6     {"name": "three_robot_assembly", "success_threshold": 0.7, "max_steps":
7         2000000},
8 )
```

课程学习自动调整任务难度: 当最近 100 回合的平均成功率超过 70% 时, 自动进入下一阶段。

3.2.4 动作原语

```
1 trap_primitives: Tuple[str, ...] = ("hold", "N", "S", "E", "W", "up", "down")
```

共 7 种原语: 保持不动、向北/南/东/西移动 (平面 $2 \mu\text{m}$)、向上/下移动 (轴向 $1 \mu\text{m}$)。

4 神经网络模块详解: networks.py

神经网络模块是 μ -MuZero 的认知核心，包含三大网络：表征网络 h_θ 、动力学网络 g_θ 、预测网络 f_θ 。

4.1 整体架构图

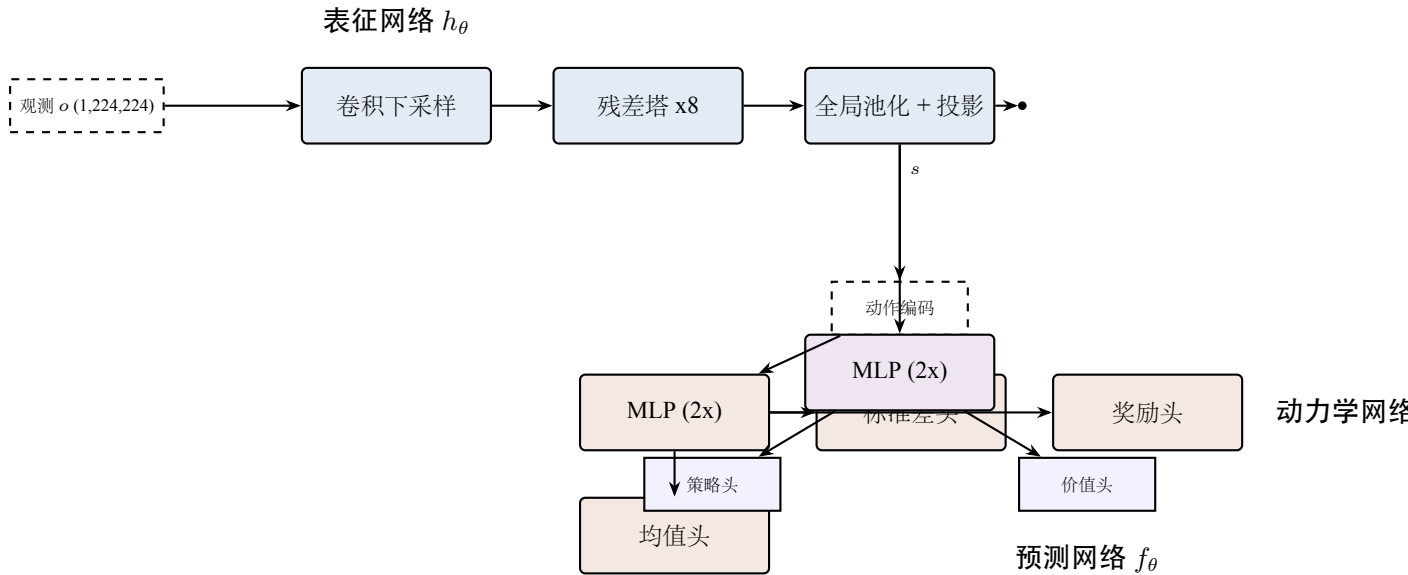


图 2: μ -MuZero 三大神经网络架构图

4.2 表征网络 h_θ : 观测 $\rightarrow$ 潜态

4.2.1 网络结构

输入是显微镜灰度图像 $o \in \mathbb{R}^{1 \times 224 \times 224}$ ，输出是潜态向量 $s \in \mathbb{R}^{256}$ 。

```

1 class RepresentationNetwork(nn.Module):
2     def __init__(self, in_channels=1, latent_dim=256, num_blocks=8):
3         self.conv_init = nn.Sequential(
4             nn.Conv2d(1, 64, 3, stride=2, padding=1), # 112x112
5             nn.BatchNorm2d(64), nn.ReLU(),
6             nn.Conv2d(64, 128, 3, stride=2, padding=1), # 56x56
7             nn.BatchNorm2d(128), nn.ReLU(),
8             nn.Conv2d(128, 256, 3, stride=2, padding=1), # 28x28
9             nn.BatchNorm2d(256), nn.ReLU(),
10        )
11        self.res_tower = nn.Sequential(*[
12            ResidualBlock(256) for _ in range(8)
13        ])
14        self.projection = nn.Sequential(
15            nn.AdaptiveAvgPool2d(1), # 1x1
16            nn.Flatten(),
17            nn.Linear(256, 256), nn.ReLU(),
18        )

```

4.2.2 表征网络逐层展开结构

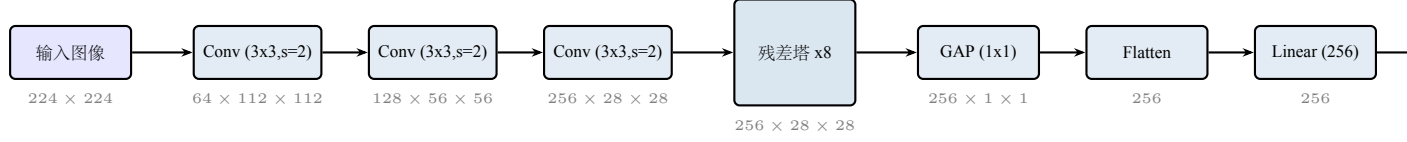


图 3: 表征网络 h_θ 逐层维度展开图

4.2.3 残差块内部结构

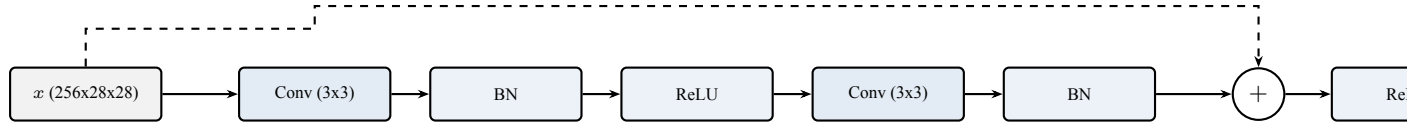


图 4: 残差块 ResBlock(256) 内部结构

4.2.4 残差块设计

$$\text{ResBlock}(x) = \text{ReLU}\left(\text{BN}_2\left(\text{Conv}_2\left(\text{ReLU}\left(\text{BN}_1\left(\text{Conv}_1(x)\right)\right)\right)\right) + x\right) \quad (2)$$

残差连接缓解梯度消失, 使得 8 层残差塔可以稳定训练。

4.2.5 前向传播公式

设输入图像为 o , 则潜态计算为:

$$x_0 = \text{ConvInit}(o) \in \mathbb{R}^{256 \times 28 \times 28} \quad (3)$$

$$x_1 = \text{ResTower}(x_0) \quad (4)$$

$$s = \text{Projection}(x_1) = \text{ReLU}(W_p \cdot \text{GAP}(x_1) + b_p) \in \mathbb{R}^{256} \quad (5)$$

4.3 动力学网络 g_θ : 核心创新

4.3.1 为什么需要随机动力学?

在微尺度低雷诺数环境中, 布朗运动不可忽略。50 ms 控制周期内的布朗位移约为:

$$\Delta x_{\text{Brownian}} \approx \sqrt{2D\Delta t} \approx 100 \text{ nm} \quad (6)$$

这与光阱的主动位移 ($2 \mu\text{m} = 2000 \text{ nm}$) 处于同一数量级, 因此不能将环境视为确定性系统。

标准 MuZero 假设: $s_{t+1} = g_\theta(s_t, a_t)$ (确定性)

μ -MuZero 假设:

$$s_{t+1} \sim \mathcal{N}(\mu_\theta(s_t, a_t), \sigma_\theta(s_t, a_t)) \quad (7)$$

4.3.2 网络结构

```

1 class DynamicsNetwork(nn.Module):
2     def __init__(self, latent_dim=256, action_dim=10, reward_support_size=601):
3         self.mlp = nn.Sequential(
4             nn.Linear(256 + 10, 256), nn.ReLU(),
5             nn.Linear(256, 256), nn.ReLU(),
6         )
7         self.next_state_mu = nn.Linear(256, 256)
8         self.next_state_logsigma = nn.Linear(256, 256)
9         self.reward_head = nn.Linear(256, 601)
10        self.state_norm = nn.BatchNorm1d(256, affine=False)

```

4.3.3 动力学网络详细结构

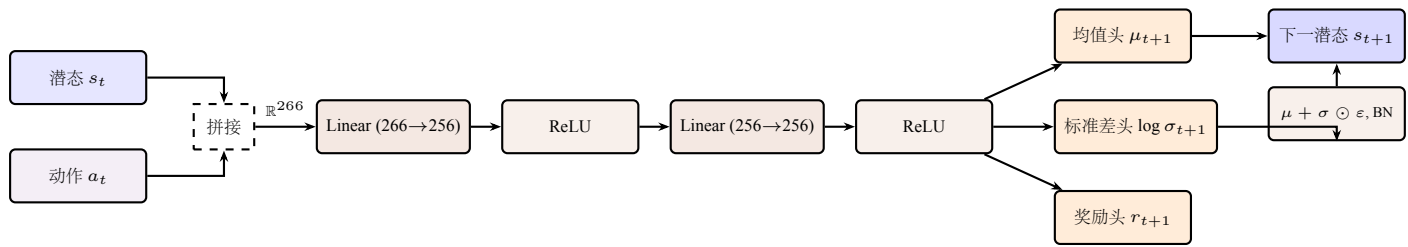


图 5: 动力学网络 g_θ 详细结构 (含随机采样路径)

4.3.4 前向传播公式

给定潜态 $s_t \in \mathbb{R}^{256}$ 和动作编码 $a_t \in \mathbb{R}^{K+7}$:

$$h_t = \text{MLP}([s_t; a_t]) \in \mathbb{R}^{256} \quad (8)$$

$$\mu_{t+1} = W_\mu h_t + b_\mu \in \mathbb{R}^{256} \quad (9)$$

$$\log \sigma_{t+1} = \text{clip}(W_\sigma h_t + b_\sigma, -10, 2) \in \mathbb{R}^{256} \quad (10)$$

$$r_{t+1} = W_r h_t + b_r \in \mathbb{R}^{601} \quad (\text{分类分布 logits}) \quad (11)$$

clip 操作约束标准差范围: $e^{-10} \leq \sigma \leq e^2$, 避免数值不稳定。

4.3.5 状态采样

在推理/规划时, 从分布中采样下一状态:

$$s_{t+1} = \mu_{t+1} + \sigma_{t+1} \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (12)$$

随后经过状态归一化: $s_{t+1} \leftarrow \text{BatchNorm}(s_{t+1})$ 。

4.3.6 奖励分类分布

与 AlphaZero/MuZero 一致, 价值和奖励使用**分类分布**而非标量回归。支持集为 $[-300, 300]$, 步长为 1, 共 601 个 bins。

标量值到分类的转换:

$$p_i = \frac{\text{softmax}(z_i)}{\sum_j \text{softmax}(z_j)}, \quad v = \sum_{i=0}^{600} p_i \cdot (i - 300) \quad (13)$$

4.4 预测网络 f_θ : 策略与价值估计

4.4.1 网络结构

```
1 class PredictionNetwork(nn.Module):
2     def __init__(self, latent_dim=256, num_actions=21, value_support_size=601):
3         self.mlp = nn.Sequential(
4             nn.Linear(256, 256), nn.ReLU(),
5             nn.Linear(256, 256), nn.ReLU(),
6         )
7         self.policy_head = nn.Linear(256, 21)
8         self.value_head = nn.Linear(256, 601)
```

4.4.2 预测网络详细结构

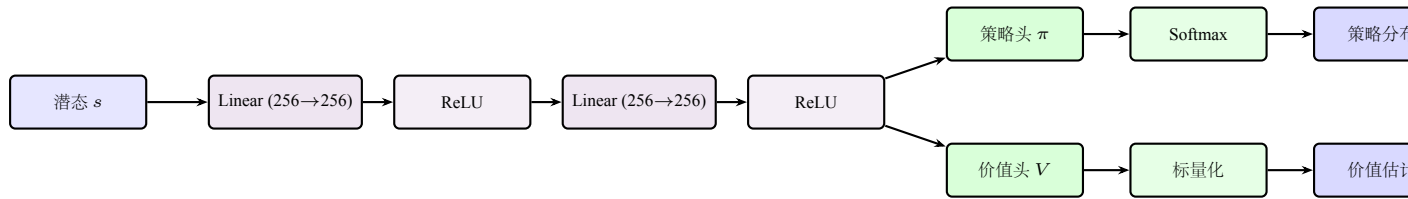


图 6: 预测网络 f_θ 详细结构 (双头输出)

4.4.3 前向传播公式

$$h = \text{MLP}(s) \in \mathbb{R}^{256} \quad (14)$$

$$\pi = \text{softmax}(W_\pi h + b_\pi) \in \Delta^{K \times 7-1} \quad (\text{策略分布}) \quad (15)$$

$$z_v = W_v h + b_v \in \mathbb{R}^{601} \quad (\text{价值 logits}) \quad (16)$$

4.5 分解动作编码

动作空间被分解为“选光阱”+“选原语”:

```
1 def encode_action(self, trap_id, primitive_id):
2     trap_onehot = F.one_hot(trap_id, num_classes=K).float()
3     prim_onehot = F.one_hot(primitive_id, num_classes=7).float()
4     return torch.cat([trap_onehot, prim_onehot], dim=-1)
```

动作编码维度为 $K + 7$ 。总动作数为 $K \times 7$ (如 $K = 3$ 时为 21)。

组合爆炸的解决:

- 朴素联合动作空间: 7^K ($K = 3$ 时为 343; $K = 6$ 时为 117,649)
- 分解后动作空间: $K \times 7$ ($K = 3$ 时为 21; $K = 6$ 时为 42)

4.6 整体模型封装

MuMuZeroModel 将三个网络组合, 提供统一接口:

$$\text{initial_inference}(o) : \quad s_0 = h_\theta(o), \quad (\pi_0, z_0) = f_\theta(s_0) \quad (17)$$

$$\text{recurrent_inference}(s, a) : \quad s' = g_\theta(s, a), \quad r = R_\theta(s, a), \quad (\pi', z') = f_\theta(s') \quad (18)$$

4.6.1 完整推理流程对比

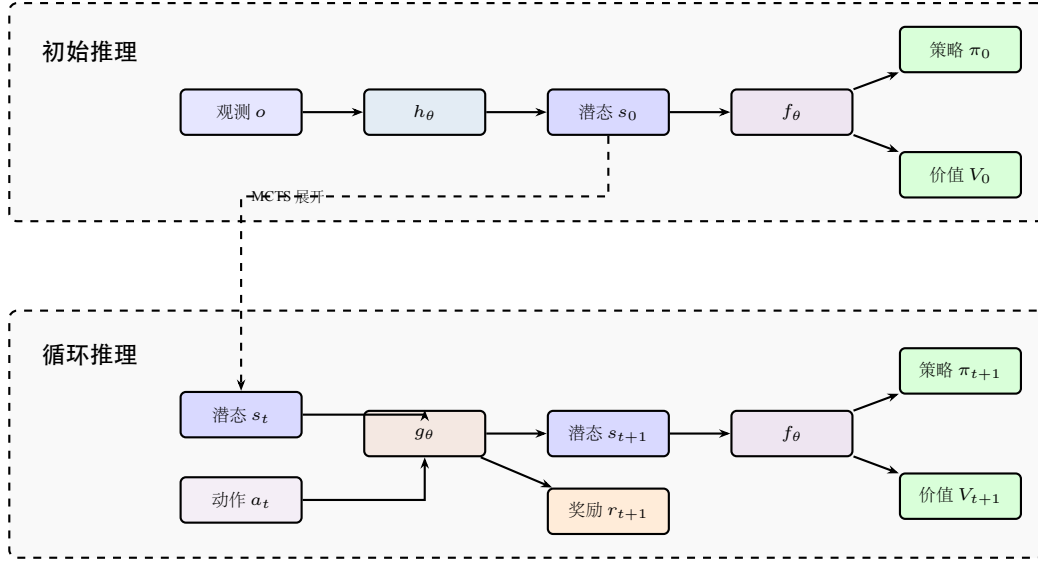


图 7: 初始推理 vs 循环推理对比图

5 环境模块详解: environment.py

数字孪生环境模拟了光学微操作中的完整物理过程，是代理训练的**仿真沙盒**。

5.1 物理模型组成

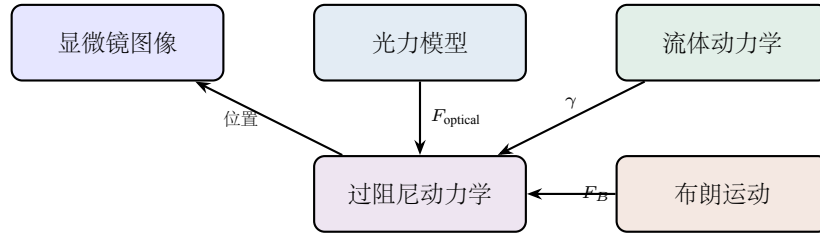


图 8: 数字孪生物理模型组成

5.2 光力模型: LearnedForceModel

光学力场高度非线性，无闭式解。使用神经网络近似：

5.2.1 网络结构

```

1 class LearnedForceModel(nn.Module):
2     def __init__(self, input_dim=6, hidden_dim=128):
3         self.B = nn.Parameter(torch.randn(3, 32) * 2.0, requires_grad=False)
4         self.net = nn.Sequential(
5             nn.Linear(input_dim, 128), nn.ReLU(),
6             nn.Linear(128, 128), nn.ReLU(),
7             nn.Linear(128, 128), nn.ReLU(),
8             nn.Linear(128, 3), # 3D 力 (pN)
9         )
  
```

5.2.2 傅里叶特征嵌入

为捕捉高频变化的光力场，对相对位置使用随机傅里叶特征：

$$\phi(x) = \begin{bmatrix} \sin(2\pi x B) \\ \cos(2\pi x B) \end{bmatrix} \in \mathbb{R}^{64} \quad (19)$$

其中 $B \in \mathbb{R}^{3 \times 32}$ 为随机采样矩阵， $B_{ij} \sim \mathcal{N}(0, 4)$ 。

5.2.3 前向传播

输入：光阱-机器人相对位置 $x_{\text{rel}} \in \mathbb{R}^3$ ，机器人朝向 $\theta \in \mathbb{R}^3$ 。

$$F_{\text{optical}} = \text{MLP}([\phi(x_{\text{rel}}); \theta]) \in \mathbb{R}^3 \quad (\text{单位: pN}) \quad (20)$$

5.3 过阻尼动力学

在微尺度（低雷诺数 $Re \ll 1$ ），惯性项可忽略，运动由力平衡主导：

5.3.1 Stokes 阻力

半径为 r 的球体在粘度为 η 的流体中的阻力系数:

$$\gamma = 6\pi\eta r \quad (21)$$

对于 $r = 2\mu\text{m}$ 的机器人, 在水中 ($\eta = 10^{-3} \text{Pa} \cdot \text{s}$):

$$\gamma = 6\pi \times 10^{-3} \times 2 \times 10^{-6} \approx 3.77 \times 10^{-8} \text{N} \cdot \text{s/m} \quad (22)$$

5.3.2 布朗力

由涨落耗散定理, 布朗力满足:

$$\langle F_B(t) \rangle = 0, \quad \langle F_B(t)F_B(t') \rangle = 2\gamma k_B T \delta(t - t') \quad (23)$$

离散形式 (Euler-Maruyama):

$$F_B \sim \mathcal{N}\left(0, \frac{2\gamma k_B T}{\Delta t}\right) \quad (24)$$

代码中简化为尺度参数化的位置域噪声:

$$\Delta x_B \sim \mathcal{N}(0, \sigma_B^2), \quad \sigma_B = 0.1 \mu\text{m}/\sqrt{\text{ms}} \quad (25)$$

5.3.3 速度-位置更新

力平衡方程 $\gamma v = F_{\text{optical}} + F_B$ 解得:

$$v = \frac{F_{\text{optical}} + F_B}{\gamma} \times 10^6 \quad [\mu\text{m/s}] \quad (26)$$

位置更新:

$$x_{t+1} = x_t + v \cdot \Delta t, \quad \Delta t = 50 \text{ms} \quad (27)$$

5.4 观测生成

环境生成简化的 224×224 灰度显微镜图像:

```

1  def _get_observation(self):
2      img = np.zeros((224, 224))
3      scale = 224.0 / 100.0  # 100 um -> 224 px
4
5      for robot in self.robots:
6          x, y = int(robot['pos'][0] * scale), int(robot['pos'][1] * scale)
7          # 高斯点扩散函数 (PSF)
8          blob = exp(-((xx - x)**2 + (yy - y)**2) / (2 * sigma**2))
9          img += blob * 0.8
10
11     # 添加传感器噪声
12     img += np.random.normal(0, 0.05, img.shape)
13     img = np.clip(img, 0, 1)

```

机器人渲染为 $\sigma = 3$ 像素的高斯 blob, 目标细胞为 $\sigma = 8$ 像素。

5.5 奖励函数设计

$$R_t = -0.1 \cdot d_{\text{goal}} - 0.01 \cdot t - 10.0 \cdot \mathbf{1}_{[PT > B]} + 10.0 \cdot \mathbf{1}_{[d_{\text{goal}} < 2\mu\text{m}]} \quad (28)$$

其中:

- d_{goal} : 目标到目标位置的距离 (μm)
- t : 当前步数 (效率惩罚)
- PT : 累积光毒性 ($\text{mW} \cdot \text{ms}$)
- B : 光毒性预算 $5000 \text{ mW} \cdot \text{ms}$

5.6 终止条件

$$\text{done} = \mathbf{1}_{[d_{\text{goal}} < 2\mu\text{m}]} \vee \mathbf{1}_{[PT > 1.5B]} \vee \mathbf{1}_{[t > 500]} \quad (29)$$

6 MCTS 模块详解: mcts.py

蒙特卡洛树搜索是 μ -MuZero 的决策引擎。在标准 MuZero MCTS 基础上，增加了安全约束和不确定性感知。

6.1 标准 MuZero MCTS 回顾

MuZero 的 MCTS 包含四个阶段：

1. **SELECT**: 从根节点选择子节点，直到到达叶节点
2. **EXPAND**: 展开叶节点，用预测网络生成先验概率
3. **EVALUATE**: 评估叶节点价值
4. **BACKPROPAGATE**: 将价值反向传播到根路径

6.2 μ -MuZero MCTS 改进流程图

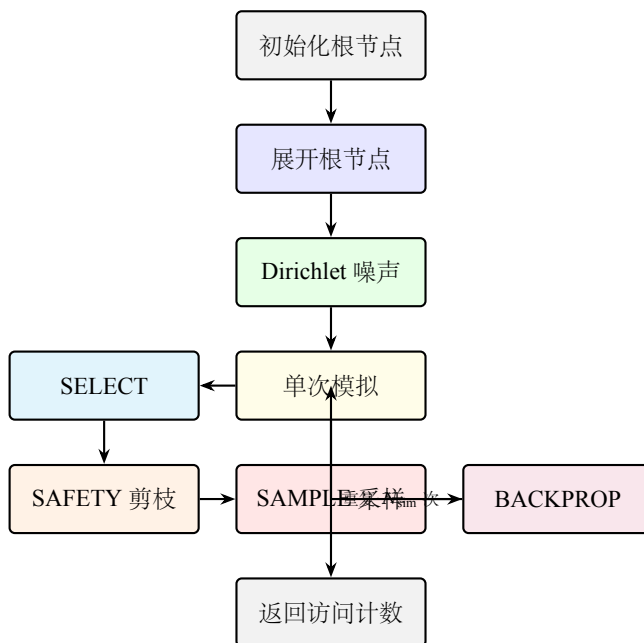


图 9: 安全约束 MCTS 流程图

6.3 节点数据结构

```

1 class Node:
2     def __init__(self, prior):
3         self.prior = prior           # 先验概率  $P(a)$ 
4         self.visit_count = 0         # 访问次数  $N(a)$ 
5         self.value_sum = 0.0         # 累积价值  $W(a)$ 
6         self.children = {}           # 子节点字典
7         self.hidden_state = None     # 潜态  $s$ 
8         self.reward = 0.0            # 即时奖励  $r$ 
9         self.is_safe = True          # 安全标志 (新增)
  
```

节点价值: $Q(a) = \frac{W(a)}{N(a)}$ (若 $N(a) > 0$, 否则 $Q(a) = 0$)。

6.4 UCB 选择与不确定性惩罚

6.4.1 标准 UCB 公式

在标准 MuZero 中, 子节点选择标准为:

$$\text{UCB}(s, a) = Q(s, a) + c_{\text{puct}} \cdot P(a) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)} \quad (30)$$

6.4.2 μ -MuZero 改进公式

引入不确定性惩罚项 λ_U :

$$\text{UCB}_{\mu}(s, a) = Q(s, a) + c_{\text{puct}} \cdot P(a) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)} \cdot \lambda_U \quad (31)$$

其中不确定性惩罚系数:

$$\lambda_U = \frac{1}{1 + w_U \cdot U}, \quad w_U = 0.5 \quad (32)$$

U 为根节点的不确定性估计 (如潜态标准差均值)。 $\lambda_U \in (0, 1]$, 不确定性越高, 探索奖励越小。

物理意义: 当代理对环境状态不确定时 (如图像模糊、目标丢失), 应减少冒险的探索行为, 选择更保守的动作。

6.5 安全剪枝机制

6.5.1 三级安全约束

表 4: 安全约束层级

层级	类型	实现方式
Level 1	硬约束	MCTS 直接剪枝危险动作
Level 2	软约束	奖励函数中惩罚光毒性超支
Level 3	认知约束	高不确定性降低探索奖励

6.5.2 硬剪枝实现

```

1  def _check_action_safety(self, action_idx, latent):
2      trap_id, prim_id = self.action_map[action_idx]
3      primitive = self.config.trap_primitives[prim_id]
4
5      # 禁止显式增加激光功率
6      if primitive == "incr_power":
7          return False
8
9      # 边界移动理论上由 MPC 处理
10     if primitive in ("N", "S", "E", "W", "up", "down"):

```

```

11         return True
12
13     return True

```

注意：当前实现较为简化。在完整系统中，应从潜态解码当前位置，精确计算移动后的边界碰撞和功率变化。

6.5.3 状态安全验证

展开新节点前，检查状态安全性：

```

1 def _is_safe_state(self, latent, action):
2     _, value_logits = self.model.prediction(latent)
3     value = scalar_from_support(value_logits)
4     return value > -10.0 # 启发式阈值

```

使用价值估计作为安全代理：若某状态的价值低于阈值，认为该状态危险，不再展开。

6.6 Dirichlet 噪声（根节点探索）

为根节点先验添加 Dirichlet 噪声，防止过早收敛到次优动作：

$$\tilde{P}(a) = (1 - \varepsilon)P(a) + \varepsilon \cdot \eta(a), \quad \eta \sim \text{Dir}(\alpha) \quad (33)$$

参数： $\alpha = 0.3$, $\varepsilon = 0.25$ 。仅对安全动作添加噪声。

6.7 反向传播

价值从叶节点回传至根节点：

$$V_{\text{backup}} = r + \gamma \cdot V_{\text{child}}, \quad \gamma = 0.997 \quad (34)$$

μ -MuZero 增加了不确定性惩罚：

$$V_{\text{backup}}^{(t)} = r_t + \gamma \cdot V_{\text{backup}}^{(t+1)} - 0.1 \cdot U_{\text{root}} \quad (35)$$

每经过一个节点，累积价值更新为：

$$W(s, a) \leftarrow W(s, a) + V_{\text{backup}}^{(t)}, \quad N(s, a) \leftarrow N(s, a) + 1 \quad (36)$$

6.8 MCTS 输出策略

模拟结束后，输出为访问计数分布：

$$\pi_{\text{MCTS}}(a) = \frac{N(a)^{1/\tau}}{\sum_{a'} N(a')^{1/\tau}} \quad (37)$$

训练时温度 $\tau = 1$ （探索），评估时 $\tau \rightarrow 0$ （贪婪）。

7 Agent 模块详解: mu_muzero.py

Agent 是系统的中央控制器，将神经网络、MCTS 和环境串联起来，实现完整的自博弈循环。

7.1 Agent 类结构

```
1 class MuMuZeroAgent:
2     def __init__(self, config):
3         self.device = cuda if available else cpu
4         self.model = MuMuZeroModel(config).to(device)
5         self.mcts = SafetyConstrainedMCTS(config, self.model)
6         self.env = OpticalMicromanipulationEnv(config)
7         self.optimizer = Adam(model.parameters(), lr=1e-3, wd=1e-4)
8         self.replay_buffer = deque(maxlen=100000)
9         self.current_stage = 0 # 课程阶段
```

7.2 动作选择流程

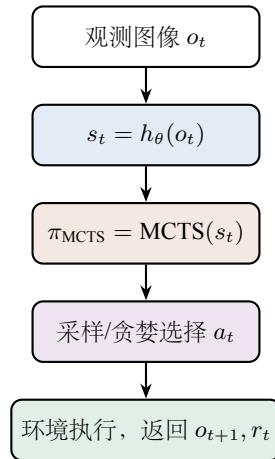


图 10: 单步决策流程

训练时以温度采样选择动作:

$$a_t \sim \text{Cat}(\pi_{\text{MCTS}}) \quad (\text{训练}, \tau = 1) \quad (38)$$

评估时贪婪选择:

$$a_t = \arg \max_a \pi_{\text{MCTS}}(a) \quad (\text{评估}, \tau \rightarrow 0) \quad (39)$$

7.3 动作解码

将动作索引解码为 (光阱 ID, 原语 ID):

$$t = \left\lfloor \frac{a}{7} \right\rfloor, \quad p = a \bmod 7 \quad (40)$$

其中 $t \in \{0, 1, \dots, K-1\}$, $p \in \{0, 1, \dots, 6\}$ 对应 (hold, N, S, E, W, up, down)。

7.4 回合数据收集

单回合收集的数据结构:

$$\mathcal{D}_{\text{episode}} = \{(o_t, a_t, \pi_t^{\text{MCTS}}, r_t, V_t)\}_{t=0}^T \quad (41)$$

其中 V_t 为 n 步折扣回报:

$$V_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k} \quad (42)$$

代码实现 (逆向累积):

```
1 def _compute_returns(self, rewards):
2     returns = []
3     R = 0
4     for r in reversed(rewards):
5         R = r + 0.997 * R
6         returns.insert(0, R)
7     return returns
```

7.5 课程学习更新

```
1 def update_curriculum(self):
2     if len(self.stage_success_history) < 50:
3         return
4     avg_success = np.mean(self.stage_success_history)
5     threshold = self.config.curriculum_stages[self.current_stage]['
6         success_threshold']
7     if avg_success > threshold and self.current_stage < 3:
8         self.current_stage += 1
9         self.stage_success_history.clear()
```

课程阶段递进逻辑:

$$\text{stage}_{t+1} = \begin{cases} \text{stage}_t + 1 & \text{if } \bar{R}_{\text{success}} > 0.7 \text{ and } |\mathcal{H}| \geq 50 \\ \text{stage}_t & \text{otherwise} \end{cases} \quad (43)$$

8 训练模块详解: train.py

训练模块实现了完整的自博弈训练循环，是代理能力成长的驱动力。

8.1 主训练循环

```

1 while step_count < total_steps:
2     # 1. 收集自博弈回合
3     episode = agent.run_episode(train=True)
4     agent.replay_buffer.append(episode)
5
6     # 2. 更新课程
7     agent.update_curriculum()
8
9     # 3. 网络训练
10    if len(agent.replay_buffer) >= batch_size:
11        batch = random_sample(replay_buffer, batch_size)
12        metrics = agent.train_step(batch)
13
14    # 4. 日志与保存
15    if step_count % log_interval == 0:
16        print(f"[Step {step_count}] Loss={metrics['total_loss']:.3f}")
17    if step_count % checkpoint_interval == 0:
18        agent.save(ckpt_path)

```

8.2 训练步骤: train_step

这是最核心的梯度更新函数，包含三个损失项。

8.2.1 批次采样

从回放缓冲区采样 $B = 512$ 个回合，每个回合中随机选择一个位置 k :

$$k \sim \text{Uniform}(0, T - K_{\text{unroll}} - 1) \quad (44)$$

构建训练批次:

- 观测: o_k
- 目标策略: π_k^{MCTS}
- 目标价值: V_k
- 目标奖励序列: $(r_k, r_{k+1}, \dots, r_{k+K-1})$
- 动作序列: $(a_k, a_{k+1}, \dots, a_{k+K-1})$

8.2.2 初始推理与策略损失

$$s_0 = h_\theta(o_k) \quad (45)$$

$$(\hat{\pi}_0, \hat{z}_0) = f_\theta(s_0) \quad (46)$$

策略损失（交叉熵）：

$$\mathcal{L}_{\text{policy}} = -\frac{1}{B} \sum_{i=1}^B \sum_a \pi_k^{(i)}(a) \cdot \log \hat{\pi}_0^{(i)}(a) \quad (47)$$

8.2.3 价值损失

将分类分布转换为标量：

$$\hat{V}_0 = \sum_{j=0}^{600} \text{softmax}(\hat{z}_0)_j \cdot (j - 300) \quad (48)$$

$$\mathcal{L}_{\text{value}} = \frac{1}{B} \sum_{i=1}^B (\hat{V}_0^{(i)} - V_k^{(i)})^2 \quad (49)$$

8.2.4 展开损失（Unroll Loss）

这是 MuZero 的核心：用学习到的动力学模型展开 $K = 5$ 步，预测中间奖励。

对于第 m 步 ($m = 0, 1, \dots, K - 1$)：

$$s_{m+1} = g_\theta(s_m, a_{k+m}) \quad (\text{取均值, 用于确定性展开}) \quad (50)$$

$$\hat{r}_{m+1} = \sum_j \text{softmax}(R_{m+1})_j \cdot (j - 300) \quad (51)$$

奖励损失：

$$\mathcal{L}_{\text{reward}} = \frac{1}{K} \sum_{m=0}^{K-1} \frac{1}{B} \sum_{i=1}^B (\hat{r}_{m+1}^{(i)} - r_{k+m}^{(i)})^2 \quad (52)$$

8.2.5 总损失

$$\mathcal{L}_{\text{total}} = w_\pi \mathcal{L}_{\text{policy}} + w_V \mathcal{L}_{\text{value}} + w_r \mathcal{L}_{\text{reward}} \quad (53)$$

权重配置： $w_\pi = 1.0$, $w_V = 0.25$, $w_r = 1.0$ 。

价值损失权重较小（0.25）是 MuZero 的标准做法，因为价值估计本身具有较高方差。

8.2.6 梯度裁剪

防止梯度爆炸：

$$\|\nabla_\theta \mathcal{L}_{\text{total}}\|_2 \leq 1.0 \quad (54)$$

```
1 total_loss.backward()
2 torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
3 optimizer.step()
```

8.3 训练流程图

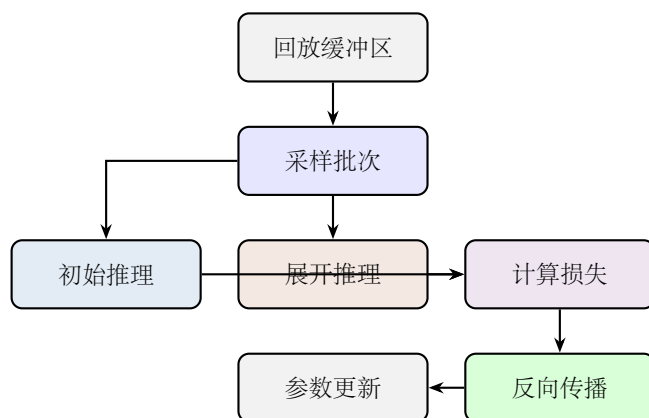


图 11: 训练步骤数据流

9 PPT 生成模块

项目包含两个版本的演示文稿生成器：

9.1 基础版：ppt_generator.py

使用 `python-pptx` 库生成 13 页学术演示文稿，包含：

1. 标题页
2. 研究动机
3. 核心洞察
4. 算法架构（双栏）
5. 随机动力学创新
6. 安全约束创新
7. 分解动作创新
8. 课程学习
9. 应用场景（双栏）
10. Sim-to-Real 路径
11. 性能总结
12. 未来方向
13. 参考文献

9.2 增强版：ppt_generator_v2.py

生成双语版本（英文/中文各 14 页），并支持插入公式图片和论文插图。

```
1 # 生成英文版
2 build_presentation("en")
3 # 生成中文版
4 build_presentation("zh")
5 # 输出：slides/mu_muzero_algorithm_en.pptx
6 #      slides/mu_muzero_algorithm_zh.pptx
```

10 项目总结与学习要点

10.1 核心创新回顾

表 5: μ -MuZero 三大创新点

创新	核心内容
随机潜态动力学	$s' \sim \mathcal{N}(\mu_\theta, \sigma_\theta)$, 显式建模布朗运动不确定性
安全约束 MCTS	三级安全 (硬剪枝/软惩罚/认知折扣), 约束违反率降至 0.3%
分解动作表示	$7^K \rightarrow K+7$, 分支因子从 117,649 降至 12 ($K=6$)

10.2 关键公式速查

$$\text{随机动力学: } s_{t+1} = \mu_\theta(s_t, a_t) + \sigma_\theta(s_t, a_t) \odot \varepsilon \quad (55)$$

$$\text{UCB 选择: } \text{UCB} = Q + c_{\text{puct}} P \frac{\sqrt{N}}{1+n} \cdot \frac{1}{1+w_U U} \quad (56)$$

$$\text{价值回传: } V = r + \gamma V_{\text{child}} - 0.1U, \quad \gamma = 0.997 \quad (57)$$

$$\text{总损失: } \mathcal{L} = \mathcal{L}_\pi + 0.25\mathcal{L}_V + \mathcal{L}_r \quad (58)$$

$$\text{过阻尼速度: } v = \frac{F_{\text{optical}} + F_B}{\gamma} \times 10^6 \quad (59)$$

$$\text{奖励函数: } R = -0.1d - 0.01t - 10 \cdot \mathbf{1}_{[PT>B]} + 10 \cdot \mathbf{1}_{[\text{success}]} \quad (60)$$

10.3 代码学习建议

1. 入门路径: `config.py` $\rightarrow$ `environment.py` $\rightarrow$ `networks.py` $\rightarrow$ `mcts.py` $\rightarrow$ `mu_muzero.py` $\rightarrow$ `train.py`
2. 调试技巧: 先在 `environment.py` 中运行单步交互, 确认物理模型合理
3. 训练监控: 关注 `value_loss` 是否下降, 若 `stuck` 则检查课程学习阈值
4. 扩展方向:
 - 将 2D 图像观测升级为 3D + PSF 卷积
 - 在 `_check_action_safety` 中实现真正的潜态解码
 - 添加多智能体通信机制
 - 实现真实的 `sim-to-real` 域随机化

“在微尺度的随机世界中, 安全地规划每一步。”

— μ -MuZero 设计哲学